

布隆过滤器

布隆过滤器（Bloom Filter）是一种用于判断元素是否属于某个集合的概率数据结构，由伯顿·霍华德·布隆（Burton Howard Bloom）于 1970 年提出。[Bloom, 1970] 它通过一系列散列函数，把元素映射到一个二进制位数组中，以较高的空间效率和查询效率完成检索。它可能产生假阳性，也就是误报；不会产生假阴性，也就是漏报。不过，简单布隆过滤器难以删除元素。

判断集合中是否存在某个元素，一种常见做法是保存所有元素，再通过比较来查找。链表、树和散列表都采用这种思路；散列表也称哈希表。三者的检索时间复杂度分别为 $O(n)$ 、 $O(\log n)$ 和 $O(1)$ 。随着集合扩大，需要的存储空间会增加，检索速度也可能变慢。布隆过滤器不保存元素本身，而是用位数组记录元素经过散列映射后留下的标记。

一个空布隆过滤器由 m 位组成，所有位最初都为 0，并配有 k 个不同的散列函数。每个函数都将元素映射到数组的某一个位置。为了取得最佳效果，这些函数应当相互独立，映射结果应服从均匀分布。通常， k 是一个较小的常数，取决于期望的假阳性率 ϵ ； m 则与 k 和准备添加的元素数量成正比。

添加元素时，分别用 k 个散列函数计算其对应的 k 个数组位置，再把这些位置全部设为 1。查询时，对待查元素进行同样的计算。如果对位位置中有任何一位是 0，就能确定该元素不在集合中：如果它曾被添加，这些位就应当全部为 1。如果所有对应位都是 1，则只能判断它可能在集合中，因为这些位也可能是在添加其他元素时碰巧被设为 1 的。这种情况会造成假阳性。简单布隆过滤器无法区分这两种情况，更先进的技术则可以解决这一问题。

以集合 $\{x, y, z\}$ 为例，设位数组长度 $m = 18$ ，散列函数数量 $k = 3$ 。在示意图中，彩色箭头表示各元素映射到的数组位置，每个元素的箭头数量对应 k 。元素 w 有一个散列位置的值为 0，因此可以确定 w 不在集合 $\{x, y, z\}$ 中。

当 k 较大时，设计 k 个不同且相互独立的散列函数可能比较困难。对于输出较宽的良好散列函数，其输出的不同位域之间应当几乎没有相关性，因此可以把输出切分成多个位域，用来生成多个“不同”的散列函数。另一种做法是使用接受初始值的散列函数，分别传入 k 个不同的初始值，例如 $0, 1, \dots, k - 1$ ；也可以把这些值加入或追加到键中。对于较大的 m 和/或 k ，可以适当放宽散列函数之间的独立性要求，由此增加的假阳性率可以忽略不计。[Dillinger 与 Manolios, 2004a; Kirsch 与 Mitzenmacher, 2006] 增强双重散列和三重散列也能有效地导出 k 个索引。这两种方法都是双散列的变体，实际上是用两个或三个散列值作为种子，驱动一个简单的随机数生成器。[Dillinger 与 Manolios, 2004b]

简单布隆过滤器无法安全地移除元素，因为它没有记录每一位被哪些元素共同使用。把目标元素对应的 k 位中任意一位清零，虽然足以让查询将该元素判为不在集合中，却也会让其他恰好映射到这一位的元素被判为不存在。由于无法判断是否有其他已添加的元素依赖这些位，清除任何一位都可能引入假阴性。

若要模拟一次性移除元素，可以增加一个辅助布隆过滤器，称为“移除过滤器”，专门记录已移除的元素。但这个方法会改变误报和漏报的边界：移除过滤器中的假阳性，会转化为原过滤器与移除过滤器共同构成的复合过滤器中的假阴性。这种方法也无法重新添加先前移除的元素，因为重新添加还需要先把它从移除过滤器中删除，于是又遇到了同样的删除难题。

布隆过滤器在空间和时间方面都具有很大优势，存储空间和插入、查询时间都是常数级的，记为 $O(k)$ 。各散列函数相互独立，便于通过硬件并行实现；由于不需要保存元素本身，它在某些保密要求非常严格的场合也有优势。

这些优势伴随着假阳性的代价：存入的元素越多，假阳性率就越高。如果元素数量很少，使用散列表已经足够。为降低假阳性率而开展的研究，也产生了许多布隆过滤器变种。当假阳性率过高时，可以重新生成过滤器。常见的情况是，所有待过滤的键都能获取，但逐一枚举的代价较高，例如需要多次读取硬盘，因此重建应当是相对少见的操作。

出处：[维基百科“布隆过滤器”](#)。依据 2026-09-10 获取的公开摘录改写，许可：CC BY-SA。